

```

classdef flgui < matlab.apps.AppBase
    % Properties that correspond to app components
    properties (Access = public)
        UIFigure          matlab.ui.Figure
        SpeedSlider        matlab.ui.control.Slider
        SpeedSliderLabel    matlab.ui.control.Label
        PausePlayButton    matlab.ui.control.Button
        TimerLabel          matlab.ui.control.Label
        StartButton        matlab.ui.control.Button
        Leaderboard         matlab.ui.control.Table
        TrackDropDown       matlab.ui.control.DropDown
        TrackDropDownLabel  matlab.ui.control.Label
        SeasonDropDown      matlab.ui.control.DropDown
        SeasonDropDownLabel matlab.ui.control.Label
        DrawTrackButton     matlab.ui.control.Button
        TrackDisplay        matlab.ui.control.UIAxes
    end
    properties (Access = private)
        Seasons = ["23", "24", "25", "26"] % Each season of F1 with
data available
        SessionID; % ID of current session being viewed
        Timer;
        StartTime; % DateTime of current session start time
        ElapsedTime; % Time delta between start time and current time
        LastUpdateTime; % Time of last update
        CurrentTime; % Time to be displayed
        EndTime; % Time of final event in session
        Drivers;
        DriverColours;
        LeaderboardEvents; % Event data i.e time at which when an
overtake happens
        NextPositionEventIndex; % Index of next event to be processed
        NextLocationEventIndex;
        DriverPositions; % Current position for each driver i.e
DriverPositions(1) = 1st
        LocationEvents; % Struct with every car position throughout
the race, sorted by time asc
        DriverLocations; % Array storing
        DriverMarkers; % Markers of each driver plotted
        FrameCount; % Total number of updates processed since sessions
start
        WallClockStart; % datetime('now') captured when Start is
pressed
        PlaybackSpeed = 1; % Speed multiplier
        IsPaused = false; % Pause state
    end
end

```

```

end
methods (Access = private)
    function data = query(app, endpoint, varargin)
        % QUERY - Handles OpenFl API requests with:
        % - Local caching
        % - Rate limiting (3/sec, 30/min)
        % - Retry logic for 429 errors
        cacheFolder = "cache/";
        if ~exist(cacheFolder, 'dir')
            mkdir(cacheFolder)
        end
        % --- Build cache key ---
        keyParts = [endpoint, string(varargin)];
        key = regexprep(strjoin(keyParts, "_"), '^[^w]', '_');
        filename = cacheFolder + key + ".mat";
        % --- Return cached data if exists ---
        if isfile(filename)
            load(filename, 'data');
            return
        end
        % --- Rate limiting ---
        persistent requestTimes
        if isempty(requestTimes)
            requestTimes = datetime.empty;
        end
        nowTime = datetime('now');
        requestTimes = requestTimes(nowTime - requestTimes <
minutes(1));
        % Limit: 3 requests/sec
        if nnz(nowTime - requestTimes < seconds(1)) >= 3
            pause(0.4);
        end
        % Limit: 30 requests/min
        if numel(requestTimes) >= 30
            waitTime = seconds(60) - (nowTime -
min(requestTimes));
            pause(max(0, seconds(waitTime)));
        end
        % --- Build URL ---
        baseURL = "https://api.openfl.org/v1/" + endpoint;
        if ~isempty(varargin)
            params = strings(1, numel(varargin)/2);
            for i = 1:2:numel(varargin)
                params((i+1)/2) = varargin{i} + "=" +
string(varargin{i+1});

```

```

        end
        url = baseURL + "?" + strjoin(params, "&");
    else
        url = baseURL;
    end
    opts =
weboptions('Timeout',15,'ContentType','json','ArrayFormat','json');
    % --- Retry logic ---
    for attempt = 1:3
        try
            requestTimes(end+1) = datetime('now');
            raw = webread(url, opts);
            if isempty(raw)
                data = [];
            elseif iscell(raw)
                raw = [raw{:}];
                data = struct2table(raw);
            elseif isstruct(raw)
                data = struct2table(raw);
            else
                data = raw;
            end
            save(filename, 'data');
            return
        catch ME
            if contains(ME.message,"429")
                pause(2 * attempt); % exponential backoff
            elseif contains(ME.message,"404")
                data = [];
                save(filename, 'data');
                return
            else
                warning(getReport(ME,"basic"));
                data = [];
                return
            end
        end
    end
    warning("Repeated rate-limit failure: " + url);
    data = [];
end
function PlotTrack(app, sessionID)
    % Uses lap 2 of a single driver to reconstruct track
layout

```

```

lapData = app.query("laps", ...
    "session_key", sessionID, ...
    "lap_number", 2);

t0 = datetime(lapData.date_start(1), ...
    'InputFormat','yyyy-MM-dd'T'HH:mm:ss.SSSSSSXXX', ...
    'TimeZone','UTC');

t1 = t0 + seconds(lapData.lap_duration(1) + 5);

% Get position samples
posData = app.query("location", ...
    "session_key", sessionID, ...
    "driver_number", lapData.driver_number(1), ...
    "date>", t0, ...
    "date<", t1);

% Remove duplicate consecutive points
changeIdx = [true; diff(posData.x) ~= 0 | diff(posData.y)
~= 0];

posData = posData(changeIdx, :);

% Smooth spline interpolation
t = 1:height(posData);
tFine = linspace(1, height(posData), 1000);

xSmooth = spline(t, posData.x, tFine);
ySmooth = spline(t, posData.y, tFine);

cla(app.TrackDisplay);
hold(app.TrackDisplay,'on');

plot(app.TrackDisplay, xSmooth, ySmooth, 'r-');

axis(app.TrackDisplay,'equal');
grid(app.TrackDisplay,'on');
end
function [dateAfter, dateBefore] = getSeasonDates(app)
    % Converts the dropdown season number to a range of dates
    % which the season was raced
    season = str2double(app.SeasonDropDown.Value);
    dateAfter = sprintf("20%d-03-01", season);
    dateBefore = sprintf("20%d-03-01", season + 1);
end
in

```

```

function UpdateFrame(app)
    % Advances simulation time based on real wall clock

    if app.IsPaused
        return;
    end
    elapsed = (datetime('now') - app.WallClockStart) *
app.PlaybackSpeed;
    app.CurrentTime = app.StartTime + elapsed;

    % Stop when race ends
    if app.CurrentTime >= app.EndTime
        stop(app.Timer);
        return;
    end

    % Update UI time
    app.TimerLabel.Text = sprintf("Current Time: %s", ...
        string(timeofday(app.CurrentTime)));

    % Update simulation state
    app.UpdateCarLocations();
    app.PlotCarLocations();
    app.UpdateLeaderboard();

    app.LastUpdateTime = app.CurrentTime;
    app.FrameCount = app.FrameCount + 1;

    drawnow limitrate;
end
function GetCarLocationEvents(app)
    % Fetch car position data in time chunks to:
    % - Avoid huge API calls
    % - Prevent timeouts
    % - Stay within rate limits

    chunkSize = minutes(10); % Size of each request window
    t0 = app.StartTime;
    t1 = t0 + chunkSize;

    totalTime = app.EndTime - app.StartTime;
    chunks = {};

    % Progress dialog
    d = uiprogessdlg(app.UIFigure, ...

```

```

        'Title','Loading', ...
        'Message','Fetching location data...', ...
        'Indeterminate','off');

while t0 < app.EndTime

    % Clamp t1 to prevent overshooting
    if t1 > app.EndTime
        t1 = app.EndTime;
    end

    % --- API CALL ---
    chunk = app.query("Location", ...
        "session_key", app.SessionID, ...
        "date>", t0, ...
        "date<", t1);

    % Store chunk
    if ~isempty(chunk)
        chunks{end+1} = chunk;
    end

    % --- Advance window ---
    t0 = t1;
    t1 = t0 + chunkSize;

    % --- Update progress bar ---
    progress = (t0 - app.StartTime) / totalTime;
    d.Value = min(progress, 1);
end

close(d);

% --- Combine all chunks ---
if isempty(chunks)
    app.LocationEvents = [];
    return;
end

% Ensure all chunks have identical columns before
concatenation
allCols = cellfun(@(c) c.Properties.VariableNames, chunks,
'UniformOutput', false);
commonCols = allCols{1};
for k = 2:numel(allCols)

```

```

        commonCols = intersect(commonCols, allCols{k},
'stable');
    end
    chunks = cellfun(@(c) c(:, commonCols), chunks,
'UniformOutput', false);

    allData = vertcat(chunks{:});

    % --- Convert timestamps ---
    times = datetime(allData.date, ...
        'InputFormat','yyyy-MM-dd'T'HH:mm:ss.SSSSSSXXX', ...
        'TimeZone','UTC');
    % Remove rows with unparseable timestamps
    validRows = ~isnan(times);
    allData = allData(validRows, :);
    times = times(validRows);
    % --- Convert to struct array (faster for playback loop)
    ---
    app.LocationEvents = struct( ...
        'Time', num2cell(times), ...
        'Driver', num2cell(allData.driver_number), ...
        'X', num2cell(allData.x), ...
        'Y', num2cell(allData.y) ...
    );

    % --- Initialise driver positions ---
    app.DriverLocations = nan(99,2);
end
function UpdateCarLocations(app)
    % Advances driver positions based on time-ordered location
events.
    % This processes all events that have occurred up to the
current simulation time.

    while app.NextLocationEventIndex <=
numel(app.LocationEvents) && ...

app.LocationEvents(app.NextLocationEventIndex).Time <=
app.CurrentTime

        % Get next location event
        ev = app.LocationEvents(app.NextLocationEventIndex);

        % Update stored position for that driver
        % (Driver number is used directly as index)

```

```

        app.DriverLocations(ev.Driver, :) = [ev.X, ev.Y];

        % Move to next event
        app.NextLocationEventIndex =
app.NextLocationEventIndex + 1;
    end
end
function PlotCarLocations(app)
    for i = 1:length(app.Drivers)
        d = app.Drivers(i);
        xy = app.DriverLocations(d,:);
        set(app.DriverMarkers(i), 'XData', xy(1), 'YData',
xy(2));
    end
end

function UpdateLeaderboard(app)
    % Processes position change events and updates leaderboard
table

    while app.NextPositionEventIndex <=
numel(app.LeaderboardEvents) && ...

app.LeaderboardEvents(app.NextPositionEventIndex).Time <=
app.CurrentTime

        % Get next leaderboard event (position update)
        ev =
app.LeaderboardEvents(app.NextPositionEventIndex);

        % Update driver at given position
        % e.g. DriverPositions(1) = driver in P1
        app.DriverPositions(ev.Position) = ev.Driver;

        % Move to next event
        app.NextPositionEventIndex =
app.NextPositionEventIndex + 1;
    end

    % Build table for UI display
    Position = (1:length(app.DriverPositions))'; %
[1;2;3;...]
    Driver = app.DriverPositions(:); % column
vector

```

```

        T = table(Position, Driver);

        % Only update UI every 10 frames (performance
optimisation)
        if mod(app.FrameCount, 10) == 0
            app.Leaderboard.Data = T;
        end
    end
function GetLeaderboardEvents(app)
    % Fetches all position change events for the session
    % and converts them into a time-sorted struct array

    data = app.query("position", ...
        "session_key", app.SessionID);

    % Convert dates to string for parsing
    dates = string(data.date);

    % Some timestamps include fractional seconds, some don't
    hasFraction = contains(dates, ".");

    % Preallocate datetime array
    times = NaT(size(dates), 'TimeZone', 'UTC');

    % Parse timestamps WITH fractional seconds
    times(hasFraction) = datetime(dates(hasFraction), ...
        'InputFormat', "yyyy-MM-dd'T'HH:mm:ss.SSSSSSXXX", ...
        'TimeZone', 'UTC');

    % Parse timestamps WITHOUT fractional seconds
    times(~hasFraction) = datetime(dates(~hasFraction), ...
        'InputFormat', "yyyy-MM-dd'T'HH:mm:ssXXX", ...
        'TimeZone', 'UTC');

    % Convert to struct array for fast sequential playback
    app.LeaderboardEvents = struct( ...
        'Time', num2cell(times), ...
        'Driver', num2cell(data.driver_number), ...
        'Position', num2cell(data.position) ...
    );
end
function GetAllDrivers(app)
    % Retrieves all drivers participating in the session

    data = app.query("drivers", ...

```

```

        "session_key", app.SessionID);

    % Store driver numbers (used throughout simulation)
    app.Drivers = data.driver_number;
end
function GetAllDriverColours(app)
    % Converts team hex colours into RGB and stores per driver

    % Preallocate (index = driver number)
    app.DriverColours = nan(99,3);

    data = app.query("drivers", ...
        "session_key", app.SessionID);

    for i = 1:height(data)
        driver = data.driver_number(i);
        hex = data.team_colour{i};    % e.g. "FF0000"

        % Convert hex → RGB
        r = hex2dec(hex(1:2));
        g = hex2dec(hex(3:4));
        b = hex2dec(hex(5:6));

        rgb = [r g b] / 255;

        % Store colour using driver number as index
        app.DriverColours(driver, :) = rgb;
    end
end
function StopAndReset(app)
    % Stops simulation and resets all app state

    % --- Stop and delete timer ---
    if ~isempty(app.Timer) && isvalid(app.Timer)
        stop(app.Timer);
        delete(app.Timer);
        app.Timer = [];
    end

    % --- Remove plotted driver markers ---
    if ~isempty(app.DriverMarkers)
        delete(app.DriverMarkers(isvalid(app.DriverMarkers)));
        app.DriverMarkers = [];
    end
end

```

```

    % --- Reset simulation variables ---
    app.SessionID          = [];
    app.StartTime          = [];
    app.CurrentTime        = [];
    app.EndTime            = [];
    app.LastUpdateTime      = [];
    app.WallClockStart      = [];
    app.Drivers            = [];
    app.DriverColours       = [];
    app.DriverPositions     = [];
    app.LeaderboardEvents   = [];
    app.LocationEvents      = [];
    app.DriverLocations     = [];

    % Reset event indices
    app.NextPositionEventIndex = 1;
    app.NextLocationEventIndex = 1;

    app.FrameCount          = 0;

    % --- Reset UI ---
    app.Leaderboard.Data = {};
    app.TimerLabel.Text = 'Current Time: ';
    cla(app.TrackDisplay);
end
end
% Callbacks that handle component events
methods (Access = private)
    % Button pushed function: DrawTrackButton
    function DrawTrackButtonPushed(app, event)
        app.PlotTrack(app.SessionID);
    end
    % Drop down opening function: SeasonDropDown
    function SeasonDropDownOpening(app, event)
        app.SeasonDropDown.Items=app.Seasons;
    end
    % Value changed function: SeasonDropDown
    function SeasonDropDownValueChanged(app, event)
        value = app.SeasonDropDown.Value;
    end
    % Drop down opening function: TrackDropDown
    function TrackDropDownOpening(app, event)
        [dateAfter, dateBefore] = app.getSeasonDates();
        data = app.query('meetings', ...
            'date_start>', dateAfter, ...

```

```

        'date_start<', dateBefore);
    if ~isempty(data)
        app.TrackDropDown.Items = data.circuit_short_name;
    end
end
% Value changed function: TrackDropDown
function TrackDropDownValueChanged(app, event)
    app.StopAndReset(); % <-- add this line at the top
    value = app.TrackDropDown.Value;
    [dateAfter, dateBefore] = app.getSeasonDates();
    data = app.query('sessions', ...
        'date_start>', dateAfter, ...
        'date_start<', dateBefore, ...
        'circuit_short_name', value, ...
        'session_name', "Race");
    app.SessionID = data.session_key;
end
% Button pushed function: StartButton
function StartButtonPushed(app, event)
    % Initialises and starts race replay simulation

    % Initialise leaderboard state (max 22 drivers)
    app.DriverPositions = zeros(1, 22);

    % Load required data
    app.GetAllDrivers();
    app.GetAllDriverColours();

    % Reset indices and frame counter
    app.NextPositionEventIndex = 1;
    app.NextLocationEventIndex = 1;
    app.FrameCount = 0;

    % Load leaderboard events
    app.GetLeaderboardEvents();

    numDrivers = numel(app.Drivers);

    % Create plot markers for each driver
    app.DriverMarkers = gobjects(numDrivers,1);

    for i = 1:numDrivers
        app.DriverMarkers(i) = plot(app.TrackDisplay, NaN,
NaN, 'o', ...
        'Color', app.DriverColours(app.Drivers(i), :), ...

```

```

        'MarkerFaceColor',
app.DriverColours(app.Drivers(i), :));
    end

    % Set initial simulation time slightly before first
meaningful event
    app.LastUpdateTime = datetime(2000,1,1,'TimeZone','UTC');

    app.StartTime = app.LeaderboardEvents(21).Time -
seconds(5);
    app.CurrentTime = app.StartTime;

    % End time extends beyond last event
    app.EndTime = app.LeaderboardEvents(end).Time +
minutes(20);

    % Load all location data
    app.GetCarLocationEvents();

    % Create timer for simulation loop
    app.Timer = timer( ...
        'ExecutionMode','fixedRate', ...
        'Period',0.1, ... % ~10 FPS update rate
        'TimerFcn', @(~,~)UpdateFrame(app) ...
    );

    % Anchor simulation to real-world time
    app.WallClockStart = datetime('now');

    % Start simulation
    start(app.Timer);
end
% Button pushed function: PausePlayButton
function PausePlayButtonPushed(app, event)
    if app.IsPaused
        % Resuming: shift WallClockStart forward by the time
spent paused
        app.WallClockStart = datetime('now') -
(app.CurrentTime - app.StartTime) / app.PlaybackSpeed;
        app.IsPaused = false;
        app.PausePlayButton.Text = 'Pause';
    else
        app.IsPaused = true;
        app.PausePlayButton.Text = 'Play';
    end
end

```

```

end
% Value changed function: SpeedSlider
function SpeedSliderValueChanged(app, event)
    app.PlaybackSpeed = app.SpeedSlider.Value;
    % Recalculate WallClockStart so CurrentTime doesn't jump
    if ~isempty(app.WallClockStart)
        app.WallClockStart = datetime('now') -
(app.CurrentTime - app.StartTime) / app.PlaybackSpeed;
    end
end
end
end
% Component initialization
methods (Access = private)
    % Create UIFigure and components
    function createComponents(app)
        % Create UIFigure and hide until all components are
created
        app.UIFigure = uifigure('Visible', 'off');
        app.UIFigure.Position = [100 100 1688 1041];
        app.UIFigure.Name = 'MATLAB App';
        % Create TrackDisplay
        app.TrackDisplay = uiaxes(app.UIFigure);
        title(app.TrackDisplay, 'Track Display')
        app.TrackDisplay.XTick = [];
        app.TrackDisplay.YTick = [];
        app.TrackDisplay.Position = [786 241 858 765];
        % Create DrawTrackButton
        app.DrawTrackButton = uibutton(app.UIFigure, 'push');
        app.DrawTrackButton.ButtonPushedFcn =
createCallbackFcn(app, @DrawTrackButtonPushed, true);
        app.DrawTrackButton.FontSize = 24;
        app.DrawTrackButton.Position = [442 954 136 39];
        app.DrawTrackButton.Text = 'Draw Track';
        % Create SeasonDropDownLabel
        app.SeasonDropDownLabel = uilabel(app.UIFigure);
        app.SeasonDropDownLabel.HorizontalAlignment = 'right';
        app.SeasonDropDownLabel.FontSize = 24;
        app.SeasonDropDownLabel.Position = [13 957 86 32];
        app.SeasonDropDownLabel.Text = 'Season';
        % Create SeasonDropDown
        app.SeasonDropDown = uidropdown(app.UIFigure);
        app.SeasonDropDown.Items = {};
        app.SeasonDropDown.DropDownOpeningFcn =
createCallbackFcn(app, @SeasonDropDownOpening, true);

```

```

        app.SeasonDropDown.ValueChangedFcn =
createCallbackFcn(app, @SeasonDropDownValueChanged, true);
        app.SeasonDropDown.FontSize = 24;
        app.SeasonDropDown.Placeholder = 'Select Season';
        app.SeasonDropDown.Position = [114 957 230 32];
        app.SeasonDropDown.Value = {};
        % Create TrackDropDownLabel
        app.TrackDropDownLabel = uilabel(app.UIFigure);
        app.TrackDropDownLabel.HorizontalAlignment = 'right';
        app.TrackDropDownLabel.FontSize = 24;
        app.TrackDropDownLabel.Position = [35 902 64 32];
        app.TrackDropDownLabel.Text = 'Track';
        % Create TrackDropDown
        app.TrackDropDown = uidropdown(app.UIFigure);
        app.TrackDropDown.Items = {};
        app.TrackDropDown.DropDownOpeningFcn =
createCallbackFcn(app, @TrackDropDownOpening, true);
        app.TrackDropDown.ValueChangedFcn = createCallbackFcn(app,
@TrackDropDownValueChanged, true);
        app.TrackDropDown.FontSize = 24;
        app.TrackDropDown.Placeholder = 'Select Track';
        app.TrackDropDown.Position = [114 902 230 32];
        app.TrackDropDown.Value = {};
        % Create Leaderboard
        app.Leaderboard = uitable(app.UIFigure);
        app.Leaderboard.ColumnName = '';
        app.Leaderboard.RowName = {};
        app.Leaderboard.Multiselect = 'off';
        app.Leaderboard.Enable = 'off';
        app.Leaderboard.FontSize = 18;
        app.Leaderboard.Position = [63 116 318 708];
        % Create StartButton
        app.StartButton = uibutton(app.UIFigure, 'push');
        app.StartButton.ButtonPushedFcn = createCallbackFcn(app,
@StartButtonPushed, true);
        app.StartButton.FontSize = 24;
        app.StartButton.Position = [460 902 100 39];
        app.StartButton.Text = 'Start';
        % Create TimerLabel
        app.TimerLabel = uilabel(app.UIFigure);
        app.TimerLabel.FontSize = 24;
        app.TimerLabel.Position = [1325 193 288 32];
        app.TimerLabel.Text = 'Current Time: ';
        % Create PausePlayButton
        app.PausePlayButton = uibutton(app.UIFigure, 'push');

```

```

        app.PausePlayButton.ButtonPushedFcn =
createCallbackFcn(app, @PausePlayButtonPushed, true);
        app.PausePlayButton.FontSize = 24;
        app.PausePlayButton.Position = [1129 183 149 52];
        app.PausePlayButton.Text = 'Pause | Play';
        % Create SpeedSliderLabel
        app.SpeedSliderLabel = uilabel(app.UIFigure);
        app.SpeedSliderLabel.HorizontalAlignment = 'right';
        app.SpeedSliderLabel.FontSize = 24;
        app.SpeedSliderLabel.Position = [807 206 74 32];
        app.SpeedSliderLabel.Text = 'Speed';
        % Create SpeedSlider
        app.SpeedSlider = uislider(app.UIFigure);
        app.SpeedSlider.Limits = [1 8];
        app.SpeedSlider.MajorTicks = [1 2 4 8];
        app.SpeedSlider.ValueChangedFcn = createCallbackFcn(app,
@SpeedSliderValueChanged, true);
        app.SpeedSlider.FontSize = 24;
        app.SpeedSlider.Position = [903 225 175 3];
        app.SpeedSlider.Value = 1;
        % Show the figure after all components are created
        app.UIFigure.Visible = 'on';
    end
end
% App creation and deletion
methods (Access = public)
    % Construct app
    function app = flgui
        % Create UIFigure and components
        createComponents(app)
        % Register the app with App Designer
        registerApp(app, app.UIFigure)
        if nargin == 0
            clear app
        end
    end
end
% Code that executes before app deletion
function delete(app)
    % Delete UIFigure when app is deleted
    delete(app.UIFigure)
end
end
end
end

```